SESIÓN 6

Enrutamiento, Navegación y Vistas

2º DAW — IES Julián Marías

Navegación en Single Page Applications (SPA)

Hasta ahora, en nuestras apps de Angular cambiábamos lo que se veía en pantalla usando **Signals** o modificando variables. Pero las aplicaciones reales tienen **URLs** diferentes según la sección en la que te encuentres. En la web tradicional, cambiar de URL significaba pedir un HTML entero al servidor y sufrir un parpadeo en pantalla. En Angular trabajamos como una **SPA (Single Page Application)**: **la página nunca se recarga**; el enrutador intercepta la barra de direcciones y sustituye los componentes en milisegundos.

Navegación Tradicional

El navegador solicita una nueva página HTML completa al servidor en cada clic. Provoca parpadeos de pantalla y recarga total del estado.

En la web clásica (PHP, HTML estático, ASP.NET clásico), cada vez que el usuario hace clic en un enlace (una etiqueta <a> normal), el navegador envía una petición al servidor, el servidor procesa un nuevo documento HTML completo y lo devuelve.

Problemas que esto genera:

- **Parpadeo blanco en pantalla:** El navegador tiene que destruir todo el DOM actual y reconstruir la página desde cero.
- **Pérdida de estado:** Si tenías datos guardados en variables de JavaScript, desaparecen en cada salto de página.
- **Ineficiencia:** Se vuelven a descargar assets repetidos como estilos CSS, imágenes de cabecera o librerías de scripts.

Navegación SPA (Angular)

El archivo HTML nunca se recarga. El enrutador intercepta la URL y sustituye dinámicamente los componentes visuales en pantalla en milisegundos.

Solo se descarga **un único archivo HTML raíz** (index.html) cuando el usuario entra a la web por primera vez. A partir de ahí, el **Angular Router** toma el control de la URL.

Ventajas de esta tecnología:

- **Sin recargas:** El archivo HTML principal nunca se vuelve a solicitar al servidor.
- **Intercepción de la URL:** Cuando el usuario hace clic en un enlace (como /tareas/5), el enrutador captura ese evento, busca qué componente debe mostrar e intercambia el contenido visual en pantalla en milisegundos sin parpadeos.
- **Preservación del estado:** Las Signals, servicios e inyecciones de dependencias siguen vivos en memoria mientras navegas entre vistas.

Anatomía de Angular Router

Dónde ir

app.routes.ts

Tabla de correspondencias que asocia URLs con componentes visuales.
Es el archivo central de configuración donde declaramos la relación "URL →Componente".

Dónde se muestra

<router-outlet />

Marcador de posición en la plantilla donde se renderiza el componente activo.
Una etiqueta HTML propia de Angular que actúa como una "pantalla en blanco" o contenedor.

Cómo ir

routerLink

Directiva HTML para navegar entre vistas sin refrescar la página.
Sustituye al atributo href estándar del HTML tradicional.

app.router.ts

```ts
import { Routes } from '@angular/router';
import { ListaTareasComponent } from './components/lista-tareas/lista-tareas.component';
import { DetalleTareaComponent } from './components/detalle-tarea/detalle-tarea.component';

export const routes: Routes = [
  // 1. Redirección por defecto al entrar a la raíz
  { path: '', redirectTo: 'tareas', pathMatch: 'full' },

  // 2. Ruta principal (Muestra la lista de tareas)
  { path: 'tareas', component: ListaTareasComponent },

  // 3. Ruta con parámetro dinámico (Muestra el detalle de una tarea)
  { path: 'tareas/:id', component: DetalleTareaComponent },

  // 4. Ruta comodín para manejar 404 (Página no encontrada)
  { path: '**', redirectTo: 'tareas' }
];
```

app.component.html

```html
<nav class="navbar">
        <a routerLink="/tareas" routerLinkActive="active"> Mis Tareas </a>
</nav>

<main class="contenido">
        <router-outlet />
</main>
```

Añade automáticamente la clase CSS 'active' al enlace cuando la ruta actual coincida con /tareas (ideal para resaltar el botón del menú en el que está el usuario).

Reserva el área de contenido principal donde se inyectarán las pantallas/componentes según la URL navegada.

Resumen: 3 pasos:

1. Definimos la lista de direcciones en app.routes.ts.
2. Colocamos <router-outlet /> en nuestro app.component.html para decirle a Angular DÓNDE queremos que se muestren las distintas pantallas.
3. Usamos routerLink en nuestros menús para viajar de una vista a otra sin refrescar la página.

Configuración Central en app.routes.ts



```
import { Routes } from '@angular/router';
import { ListaTareasComponent } from './components/lista-tareas/lista-tareas.component';
import { DetalleTareaComponent } from './components/detalle-tarea/detalle-tarea.component';

export const routes: Routes = [
{ path: '', redirectTo: 'tareas', pathMatch: 'full' },
 { path: 'tareas', component: ListaTareasComponent },
{ path: 'tareas/:id', component: DetalleTareaComponent }, // Parámetro dinámico
{ path: '**', loadComponent: () => import('./components/not-found/not-found.component') }
];
```

Importa el tipo Routes de Angular para validar que el array de configuración cumple la estructura estricta del enrutador.

Array Routes: Es un array de objetos JavaScript donde cada objeto define una regla.
path: La porción de URL que Angular busca en la barra de direcciones.
component: El componente Standalone que se inyectará dentro del <router-outlet /> cuando la URL coincida.
pathMatch: 'full': Imprescindible en redirecciones desde la ruta vacía '' para evitar coincidencias parciales infinitas.
El segmento :id es una variable dinámica. Permite capturar URLs como /tareas/1, /tareas/42, etc., y cargar el componente de vista detalle.
El comodín ** atrapa cualquier URL que no coincida con las anteriores (páginas no encontradas o 404). En lugar de usar component, usa loadComponent: () => import(...). Esto hace que el código del componente 404 solo se descargue si el usuario comete un error en la URL

Recepción Directa de Parámetros con withComponentInputBinding()

Capacidad de vincular parámetros de la URL directamente a las propiedades @Input() o input() del componente.
Antes había que inyectar el servicio ActivatedRoute y suscribirse a un Observable (this.route.params.subscribe(...)) para leer un simple ID de la URL. Con esta función, el ID se inyecta de forma transparente directamente en el componente.

Importamos la función helper withComponentInputBinding junto con el proveedor del Router.
Al pasar withComponentInputBinding() como segundo argumento de provideRouter(), le decimos al enrutador de Angular: "Si una ruta define un parámetro dinámico como :id, busca una entrada id en el componente de destino y asígnale el valor automáticamente".

```
// src/app/app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
        providers: [ provideRouter(routes, withComponentInputBinding()), //
        Binding activado provideHttpClient()
] };
```

¿Por qué utilizarlo?

- **Sin ActivatedRoute: Ya no hace falta inyectar el servicio de ruta para suscribirse a params, ni importar ActivatedRoute, ni usar inject(ActivatedRoute), ni manejar suscripciones RxJS solo para leer un parámetro de la barra de direcciones.**
- **Integración con input(): El parámetro :id de la URL pasa directamente a un id = input.required().**
- **Código más limpio: Simplifica drásticamente los componentes de vista detalle.**

Si la ruta es /tareas/:id, en nuestro componente de vista detalle declaramos simplemente:
id = input.required<string>(); // ¡Angular le asigna el parámetro de la URL solo!
El nombre del input() dentro del TS debe coincidir exactamente con el nombre de la variable definida tras los dos puntos en app.routes.ts (:id →id).

Consumiendo Parámetros de Ruta con input()

```
import { Component, OnInit, inject, input, signal } from '@angular/core';
import { TareasService } from '../services/tareas.service';
import { Tarea } from '../interfaces/tarea.interface';

@Component({
  selector: 'app-detalle-tarea',
  standalone: true,
  templateUrl: './detalle-tarea.component.html'
})
export class DetalleTareaComponent implements OnInit {
  private tareasService = inject(TareasService);

  // 1. Recibe automáticamente el parámetro :id de la URL
  id = input.required<string>();

  // 2. Estado local para guardar los datos recibidos del backend
  tarea = signal<Tarea | null>(null);

  ngOnInit(): void {
    // 3. Petición HTTP GET al backend usando el ID recibido
    this.tareasService.obtenerTareaPorId(Number(this.id())).subscribe({
      next: (data) => this.tarea.set(data),
      error: (err) => console.error('Error al cargar la tarea:', err)
    });
  }
}
```

- **Captura automática del :id de la URL:**id = input.required<string>(); Gracias a withComponentInputBinding(), en cuanto el usuario entra a /tareas/5, Angular asigna la cadena '5' a la señal id automáticamente.
- **Estado local para la tarea:**tarea = signal<Tarea null |>(null); Inicializamos una señal en null para almacenar los datos de la tarea cuando regresen de la red.
- **Consulta asíncrona al arrancar (ngOnInit):** Pasa el ID a número (Number(this.id())), llama al método obtenerTareaPorId() del servicio y guarda el resultado en la señal tarea mediante .set(data).

Navegación Declarativa vs. Programática

Declarativa (HTML)

```
<a [routerLink]="['/tareas', tarea.id]"> Ver detalle a>
```

Es la opción ideal para elementos visuales estáticos o interactivos directos, como botones de un menú de navegación, enlaces en tablas o tarjetas de un listado. En resumen, ideal para enlaces visibles en menús, tablas o tarjetas.

- Usamos [routerLink] (con corchetes) cuando la ruta incluye variables dinámicas (como tarea.id).
- Le pasamos un array de segmentos: ['/tareas', tarea.id] se transforma en la URL /tareas/5.
- No requiere escribir ni una sola línea de lógica en el archivo

Programática (TypeScript)

```
private router = inject(Router); guardarYVolver() { this.router.navigate(['/tareas']); }
```

Es imprescindible cuando la navegación depende del resultado de una acción previa en código (ej. tras pulsar "Guardar", recibir confirmación de la API y querer volver al listado). En resumen, ideal tras completar una acción (ej. guardar un formulario).

- Se inyecta el servicio Router de @angular/router mediante inject(Router).
- Se llama al método this.router.navigate(['/ruta']) pasándole el array de segmentos de destino.

Hoja de Ruta: Multi-Pantalla en Task Tracker



1

Configurar Rutas

Crear app.routes.ts definiendo rutas principal y detalle.

2

Activar Binding

Añadir withComponentInputBinding() en app.config.ts.

3

Crear Vista Detalle

Crear componente que reciba id como input.required().

4

Enlazar Vistas

Añadir routerLink en el listado para navegar al detalle.

¿PREGUNTAS?

¡Manos a la obra en el aula!

Siguiente paso: convertir nuestra aplicación de tareas en una SPA completa con navegación multi-pantalla.